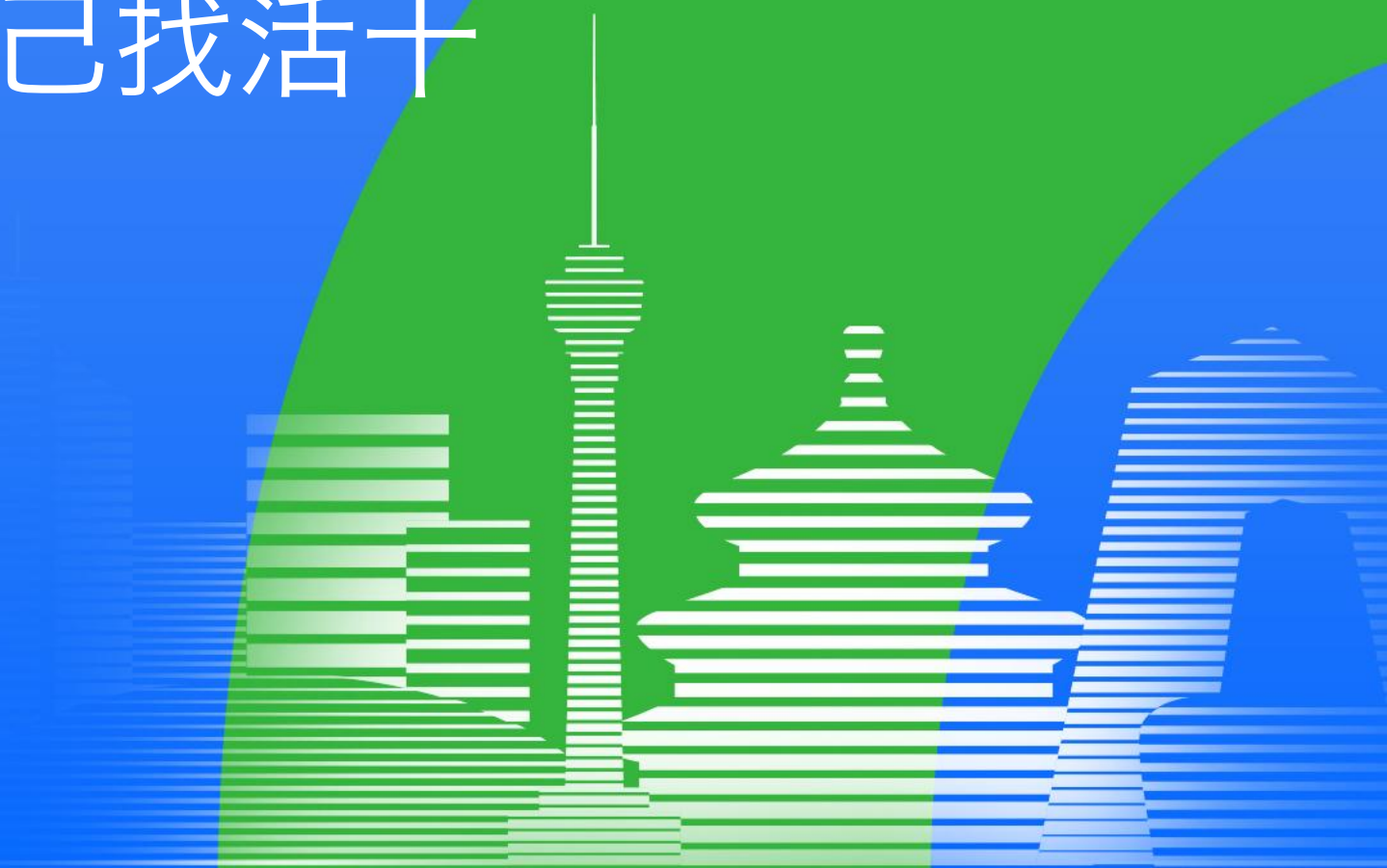


QCon
全球软件开发大会

超级团队进行时： 从用 AI 到 AI 自己找活干

victorhuang



极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

🕒 6月 📍 上海

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：6月26-27日

- AI Infra 系统工程
- 多 Agent 协作与实践
- 多模态融合
- 模型训练与推理创新
- 数据平台与特征服务

🕒 8月 📍 深圳

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：8月21-22日

- Agentic AI
- 轻量化与高效推理
- 多模态应用
- AI + IoT 场景实践
- AI 工业化落地

🕒 10月 📍 上海

👥 1200人

QCon

全球软件开发大会

会议时间：10月22-24日

- AI Agent
- Vibe Coding
- 智能可观测
- 推理基建
- 模型攻防
- AI x 创造力

🕒 12月 📍 北京

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：12月18-19日

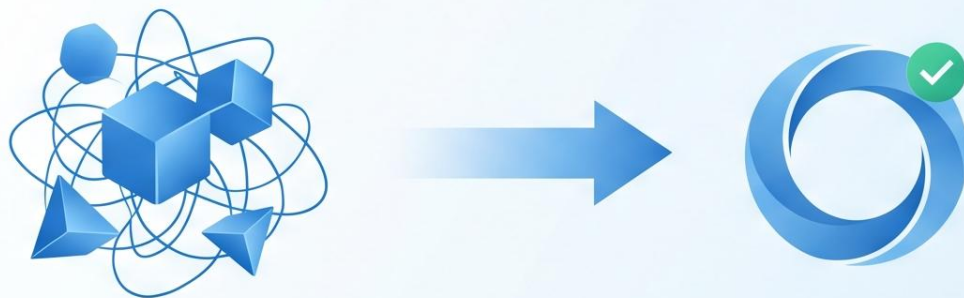
- 大模型架构创新
- 多模态 AI 产业融合
- 具身智能
- AI for Science
- 大模型安全

从用 AI 到 AI 自己找活干

中间隔了四道坎：不去用、用太少、不动脑、太烧脑



越有经验越不去用



从 20% 到 100%

1

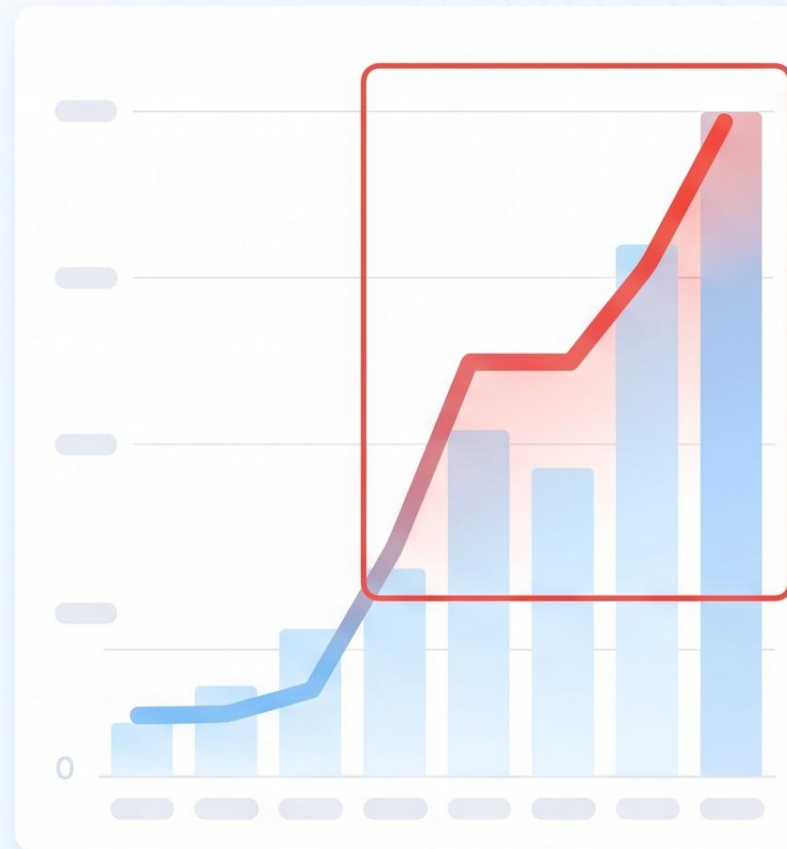
推力 1: 中山线下会, 拉起紧迫感

2

推力 2: 每周闪电分享, 沉淀近 300 个输出

3

推力 3: 模型本身在变强, 尤其 Claude 新版本本



高覆盖 ≠ 高融入

< 1h / 天

人均 AI 使用时长



原因分析：陈旧的流程与保守的工作边界



① 材料因

- Skills hub / Prompt hub 无法复用
- 别人封装的最佳实践，到你手里就是半成品



② 动力因

- 螺丝钉员工把边界锁死了
- 别人流程里的痛点与我无关



③ 目的因

- 团队总以"提效"为目的
- 效率是起点，不是终点



④ 结构因

- 大量"黑箱时间"
- AI 交付了，在等人接

拓展目的：带团队往下游看，而不是只看自己这一段

1

思考下游痛点



2

谁最痛，谁最容易成为突破口



3

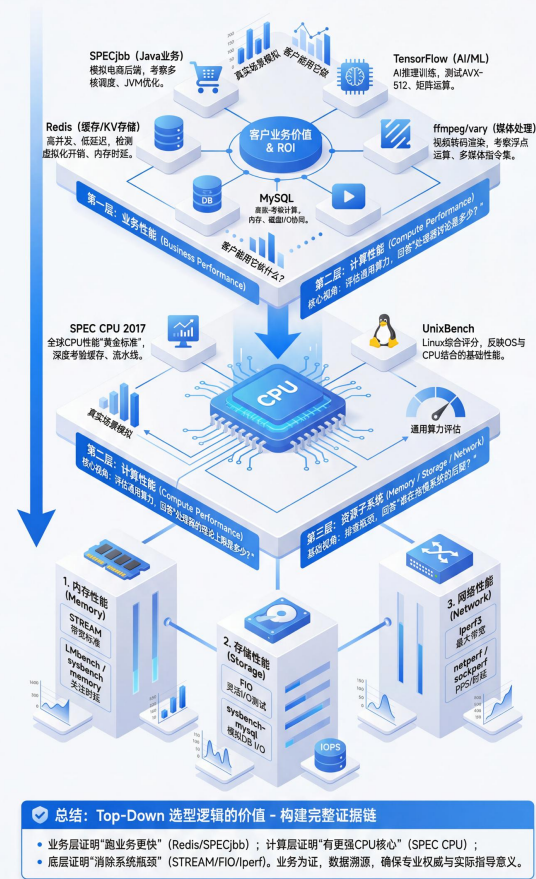
开始解决别人流程里的问题

AI 才会从个人外挂变成团队杠杆

案例：性能数据变成“弹药”

基于“Top-Down”（自顶向下）的服务器测评体系分析报告

从最终用户的业务价值出发，逐层向下剥离，深入到系统综合能力，最后落地到核心硬件组件的微观指标。



CPU 性能优化全景图

核心维度全方位调优指南

优化维度	核心影响因素	具体影响说明	评估/检测方法	优化建议
1. 硬件与微架构	CPU 代际、频率、SMT、拓扑	指令效率差异、频率波动、NUMA 延迟	'lscpu', 'turboast', 'numactl'	选最新代际、高频实例，优化 NUMA
2. 虚拟化与云实例	vCPU 类型、虚拟化开销、绑定	共享干扰、上下文切换、缓存失效	'vsteal', 'perf stat', 'taskset'	选独享型/裸金属，实施 CPU 亲和性
3. 系统与内核	调度器、安全补丁、大页、cgroups	任务迁移、KPTI 开销、TLB 未命中	'cpupower', 'vulnerabilities', 'vmstat'	Performance 模式、HugePages、cpuset
4. 工作负载特性	计算/内存/IO 受限、并行度与锁	瓶颈差异、线程切换、锁竞争等待	'perf stat' (IPC), 'perf lock', 'iostat'	向量化、优化数据结构、异步 IO
5. 编程语言/运行时	语言特定开销 (GIL, GC)	影响 CPU 执行效率	'jstack', 'pprof'	调优 GC、设置并发参数、高优化编译

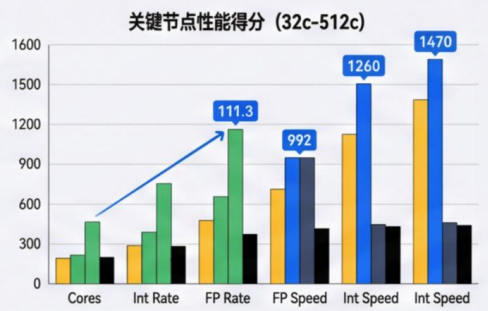
1. 硬件与微架构层面	影响因素	具体影响说明	评估/检测方法	优化建议
	CPU 代际与微架构	指令执行效率、功能支持差异	'lscpu', 'SPECint', 'sysbench'	选最新代际，确认指令集 (AVX-512)
	核心频率与加速功能	速度/功耗限制导致频率波动	'turboast', 'msr-tools', '压力测试'	良好散热，选高基础频率实例
	SMT/超线程与核心拓扑	资源竞争、NUMA 延迟	'numactl -H', SMT 对比	低延迟禁用 SMT，绑定进程优化 NUMA
2. 虚拟化与云实例形态	影响因素	具体影响说明	评估/检测方法	优化建议
	vCPU 类型	共享型干扰、突发型受限	监控 'vsteal', 'vmstat', 'top'	选独享/计算优化型，避免突发型
	虚拟化开销	指令转换、上下文切换开销	'vdistat-w', 'perf stat', '黄金对比'	追求极致选裸金属/专属宿主机
	vCPU 拓扑与绑定	调度迁移导致缓存失效、远端内存访问	'taskset' 绑定实验, 'perf top' 缓存分析	实施 CPU 亲和性，中断屏蔽
3. 系统与内核配置	影响因素	具体影响说明	评估/检测方法	优化建议
	调度器与 Governor	任务迁移增加缓存缺失，节能策略影响频率	'cpupower', 'perf' 任务迁移分析	设置 performance 模式，减少迁移
	安全缓解措施	补丁 (KPTI) 带来性能开销	查看 'sysdevices/syst' em/cpu/vulnerabilities'	评估选择性禁用部分缓解措施
	HugePages	TLB 未命中，配置不当引发抖动	'perf mem' TLB 分析, 'vmstat'	为大页应用配置 HugePages
	容器 cgroups	配置引发节流，cpuset 影响亲和性	查看 cgroup 节流时间	低延迟避免严格配置，用 cpuset 绑定
4. 工作负载特性	影响因素	具体影响说明	评估/检测方法	优化建议
	计算/内存/IO 受限	瓶颈不同优化方向各异	'perf stat' IPC 分析, 'iostat'	计算：SIMD 向量化内存：优化数据结构 IO：异步 IO
	并行度与锁竞争	线程过多切换频繁，锁竞争导致等待	'perf lock' 分析，应用剖析	线程粒度合理调优，用无锁/细粒度锁
5. 编程语言/运行时	影响因素	具体影响说明	评估/检测方法	优化建议
	不同语言特定开销 (GIL, GC)	影响 CPU 效率	语言特定工具 ('jstack', 'pprof')	JVM: 调优 GC/堆 Go: 设置 'GOMAXPROCS' Python: C 扩展 C/C++: 高优化编译

SA9e 处理器性能扩展性深度分析

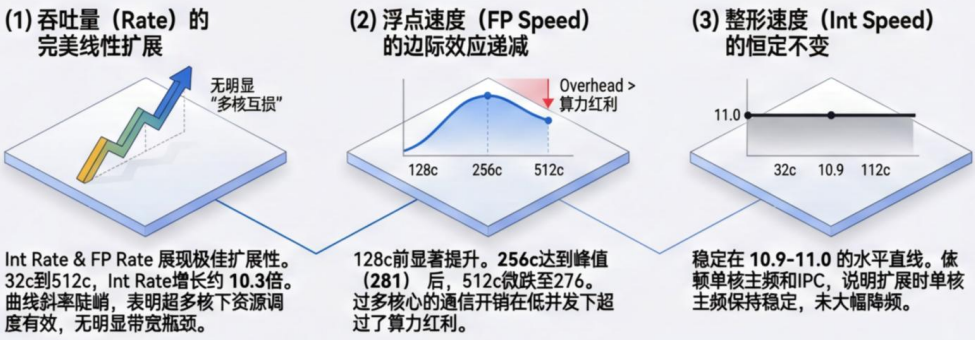
基于 SPEC CPU 2017 关键数据 (4c-512c)

1. 关键数据提取

核心数	Int Rate (整形吞吐-黄)	FP Rate (浮点吞吐-绿)	FP Speed (浮点速度-蓝)	Int Speed (整形速度-黑)
32c	142	143	108	10.9
48c	142	143	108	10.9
64c	108	178	134	10.9
96c	258	250	276	10.9
128c	748	558	276	10.9
256c	1470	1220	281	10.9
512c	1470	1260	276	10.9



2. 趋势分析



3. 总结

为高并发吞吐量而生

扩展性极强：Rate测试线性扩展至512核无明显性能瓶颈，适用于云计算、虚拟化容器和大规模数据处理。

调度优化卓越

512核下整形吞吐高达1470，证明内部互连和内存子系统带宽极其充裕。

单线程性能稳定

在堆核的同时，Int Speed未缩水，保证了基础任务的延迟表现。

增加动力：AI 内部应用最大的陷阱是“强大的 DEMO”

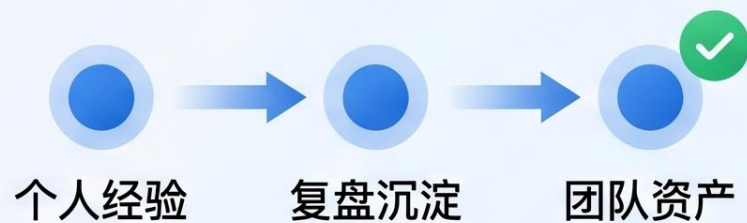
陷阱：强大的 DEMO

- 看起来什么都能做
- 实际落地一堆问题



真正有用的

- 务实的“坏消息”分享
- 每天新鲜滚烫的 AI 复盘会
- 把“某个人会”变成“团队可复用”
- 把临场判断变成上下文资产



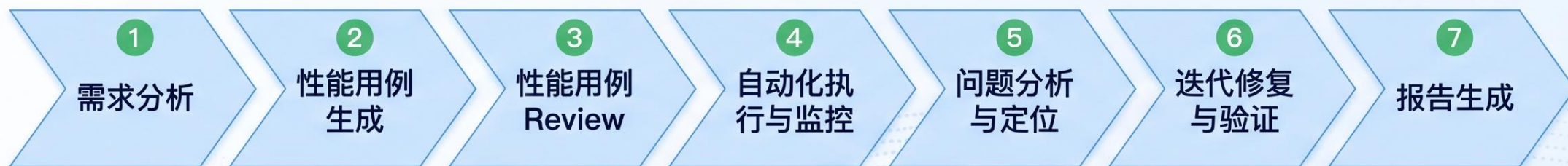
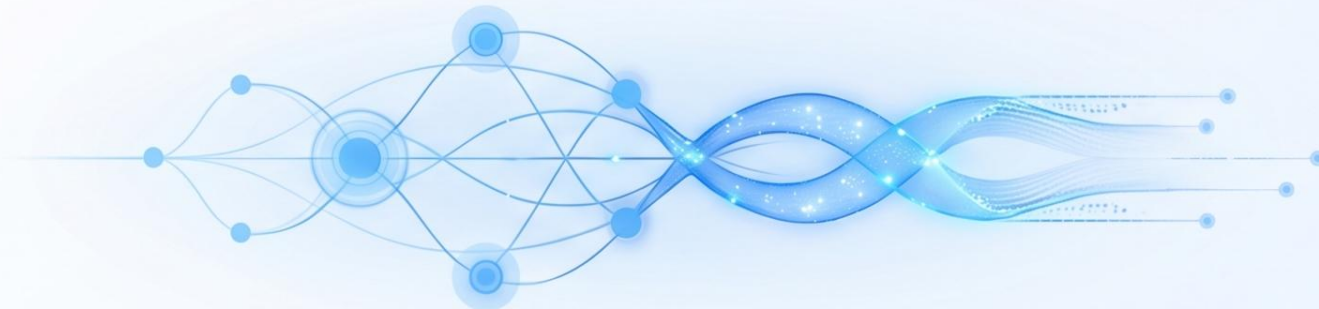
团队上下文工程设计，其实也是 AI 流程的设计



没有流程，AI 只能零散帮忙

有流程，AI 才能真正接手链路

案例：基于 CodeBuddy 的性能测试全链路



真正提效的不是某个节点自动化，而是整条链路减少人工接力

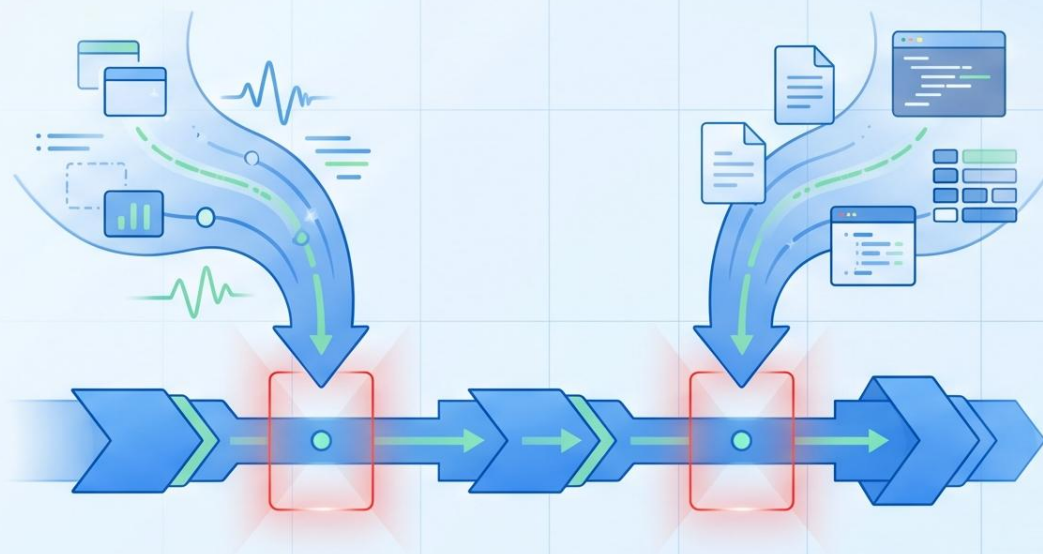
动态上下文:

- 运行时 Profile / Tracing / Logging / Metrics

静态上下文:

- 需求 / 文档 / 代码库的时间信息和空间信息

精雕细琢 = 把动态和静态上下文都接进链路



阶段性效果：从 1h 到 4h

1h →



人均 AI 使用时长

- 从“偶尔帮忙”，从广使熹的擢升电话用恹，进行人均偶尔使用时长

→ “默认工作方式”

4h



人均 AI 使用时长

默认工作方式是工作方式，组合持续化的速度他，发展实成工作式时长

使用方式转变

流程化把偶尔用，变成持续用

小结：组织开始突破边界，AI 才不再是个人外挂

突破边界

AI 从个人外挂
变成团队产能

- 给一个“用”的目的和动力



- 野心越大，焦虑越少



- 因为大家开始看到可复制的增长路径



用深了，但动脑了吗？

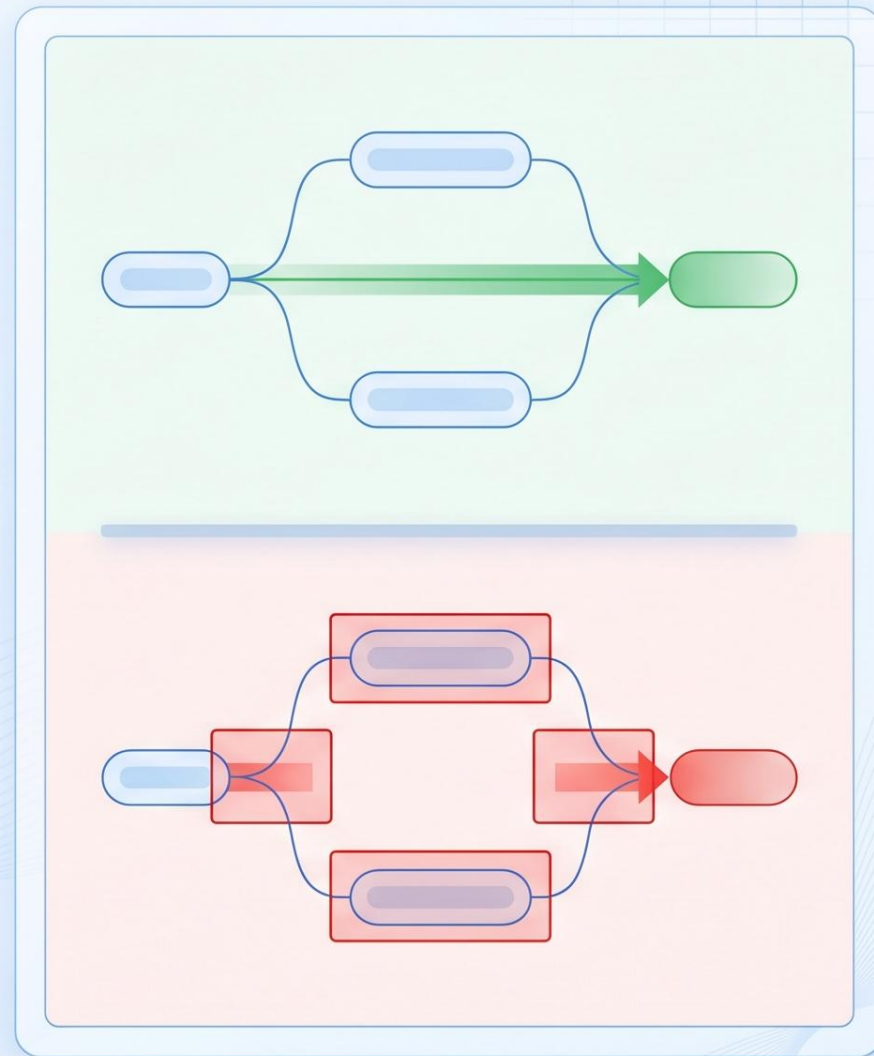
→ 用深了，但动脑了吗？

懒得动脑，战术勤奋

当下 AI 时代，
写代码比定义"好"更简单

没有期望答案 → 产生"AI 这个答案很好"的幻觉

概率模型天然补全一个"看着像对的答案"

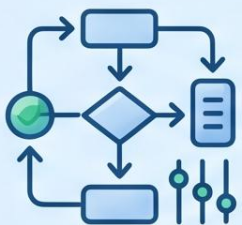


常见的假方案：继续卷提示词、调流程、换模型、堆外壳

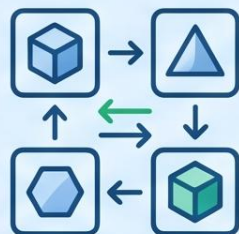
默认前提：AI 已经知道什么叫好



1 继续卷提示词



2 继续调流程

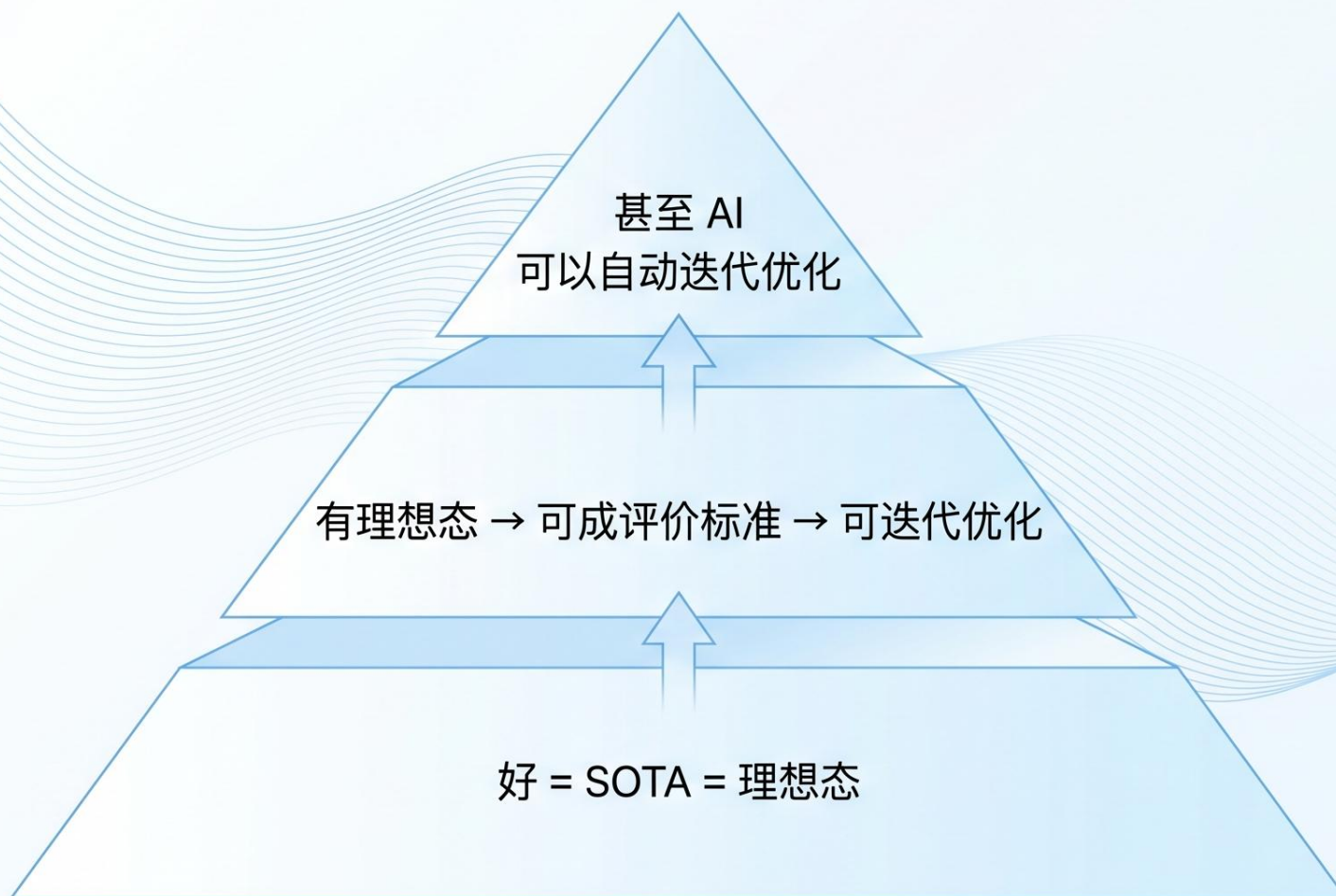


3 继续换模型



4 继续堆外壳

真问题不是"方法不够", 而是"什么算好"没有定义



- 没有理想态, 所有迭代都只是在"差不多"里打转
- 没有理想态, 一切 AI 应用都没有未来

1. 具象到抽象

- 懂业务的人不会提炼理想态
- 需要归纳总结的高级能力



2. 抽象到具象

- 理想态变成可稳定执行的评价标准
- 需要耐性，不怕脏



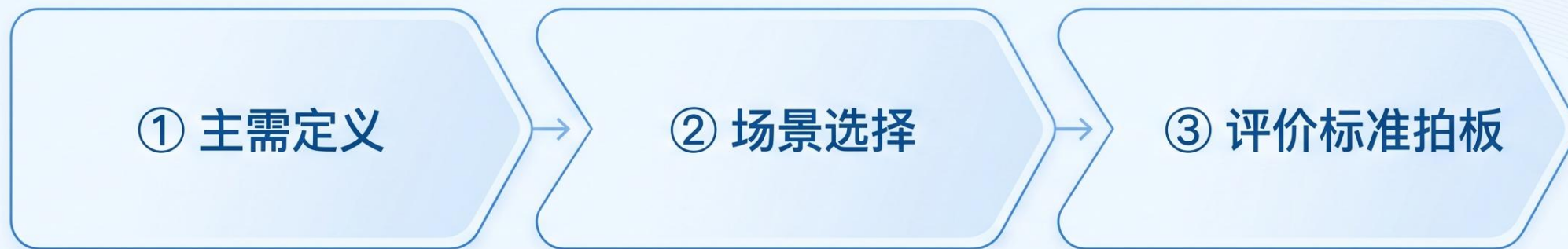
3. 合在一起

有高级能力，同时又不怕脏
这是什么鬼要求



不成熟的解法：meta-agent（从理想态生成到迭代优化的全流程 AI 赋能）

解法 1：让最懂业务的人先定义主需、场景和评价标准



✓ 谁知道真实世界怎么判分，谁就先来定义 rubric

📖 好的定义源于道与法：能成为理想态的，必输对应领域有道与法的好书

解法 2：脏活就需要管理者重视，亲自抓亲自看

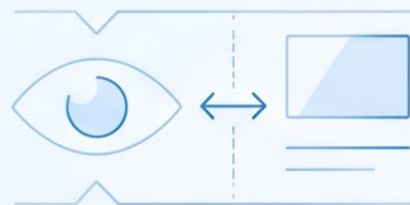
重视打分

- 🎯 建 benchmark
- 🎯 建准召率目标
- 📋 建通过率目标
- 📋 建 Design Review 和验收标准



重视洗脑

- ❓ 离理想态差在哪？为什么差？
- 👁️ 管理者要亲自看，打开看
- ↔️ 不只看这次成不成，还看跟理想态的差距
- 🔄 对于未来的定义，看看 CLBench



理想态是不是理想 理想有没有落实到用例中

挑战:

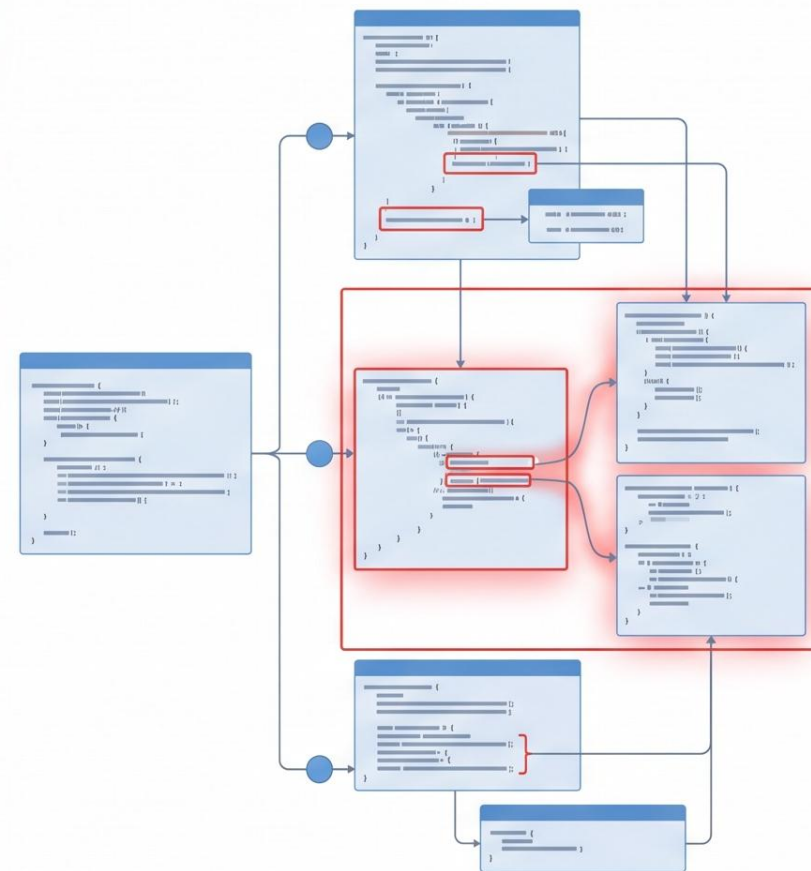
代码三态

全局变量关联

多线程问题分析

过程理想态 +
结构理想态都要有

10分 → 90分



					repomind			+repomind 后提升值		
基准场景	场景	准确率	召回率	F1	准确率	召回率	F1	准确率提升值	召回率提升值	F1提升值
	commit代码定位	67.26%	75.05%	70.94%	81.91%	89.69%	85.61%	+14.65% ↑	+14.64% ↑	+14.67% ↑
	错误码分析	63.78%	85.78%	73.17%	67.80%	93.55%	78.62%	+4.02% ↑	+7.77% ↑	+5.45% ↑
	函数调用链	86.09%	89.44%	87.72%	89.20%	93.93%	91.50%	+3.11% ↑	+4.49% ↑	+3.78% ↑
	配置参数	60.55%	86.65%	71.29%	86.66%	88.66%	87.64%	+26.11% ↑	+2.01% ↑	+16.35% ↑
	函数历史	61.49%	83.43%	70.80%	77.92%	89.55%	83.33%	+16.43% ↑	+6.12% ↑	+12.53% ↑
	异常日志分析	82.00%	92.00%	86.71%	91.00%	93.04%	92.01%	+9.00% ↑	+1.04% ↑	+5.30% ↑
	配置参数分析	86.00%	100.00%	92.47%	93.60%	98.24%	95.86%	+7.60% ↑	-1.76% ↓	+3.39% ↑
业务应用场景	用例脚本生成	50.00%	54.00%	51.92%	82.00%	80.00%	80.99%	+32.00% ↑	+26.00% ↑	+29.07% ↑
	需求分析	70.00%	63.00%	66.00%	75.00%	77.00%	76.00%	+5.00%	+12.00%	+10.00%
	性能劣化问题分析	64.00%	53.00%	58.00%	76.00%	83.00%	79.00%	+12.00%	+30.00%	+21.00%

小结：理想态不是锦上添花，而是一切的前提

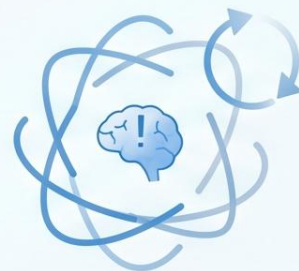
有理想态

- » 质量、速度、自动化、规模化都有方向
- » 评价标准可迭代
- » 反馈回路可闭合



没有理想态

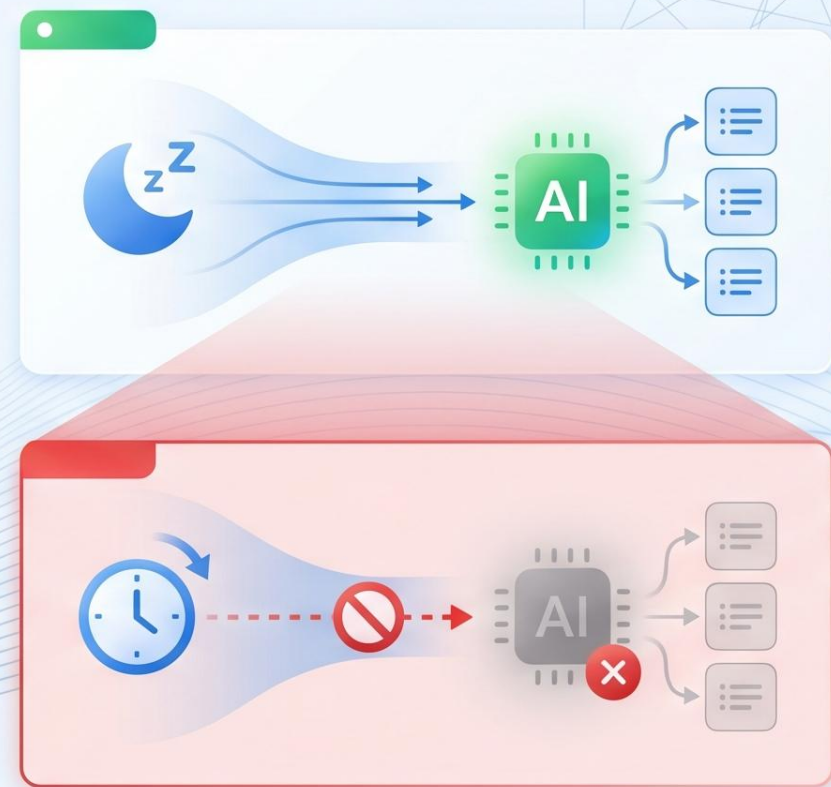
- 所有迭代在"差不多"里打转
- 没有真正可自动闭合的反馈回路
- 被迫动更多脑
需要动脑 → 现在不动，后面被迫动更多





动脑了，但太烧脑了

- 理想态：高并发 + "睡后"工作
- 现实：周末、假日、晚上 → AI 停了
- 有进展，但远远不够
- 三座大山挡在中间

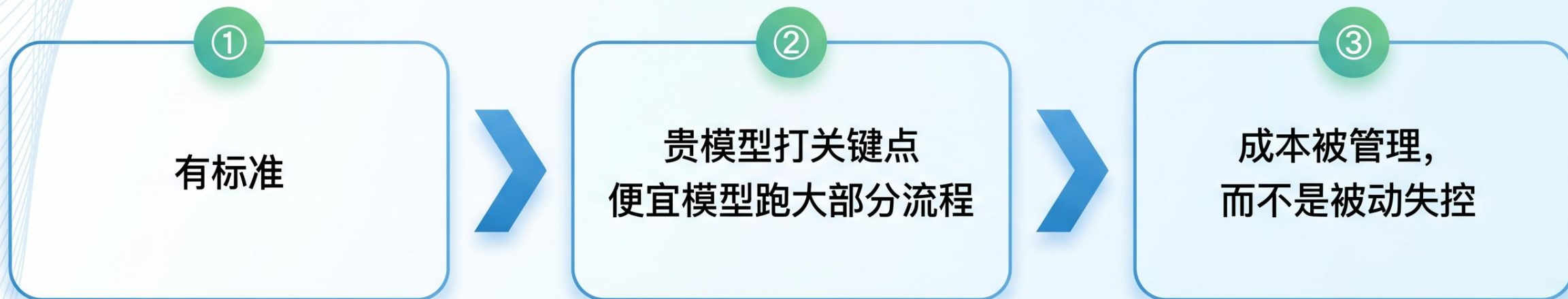


第一座山：成本和可靠性，总断总停



**长程运行不是提效，
是昂贵地犯错**

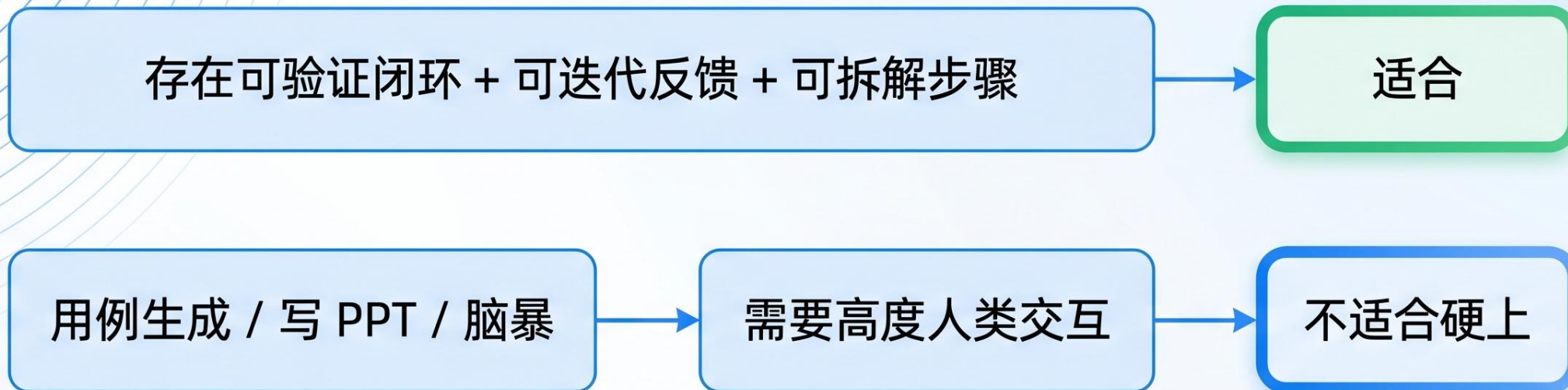
第一座山解法：用理想态和评估体系，换低成本模型和稳定收益



Harness 可以帮助路由和管理

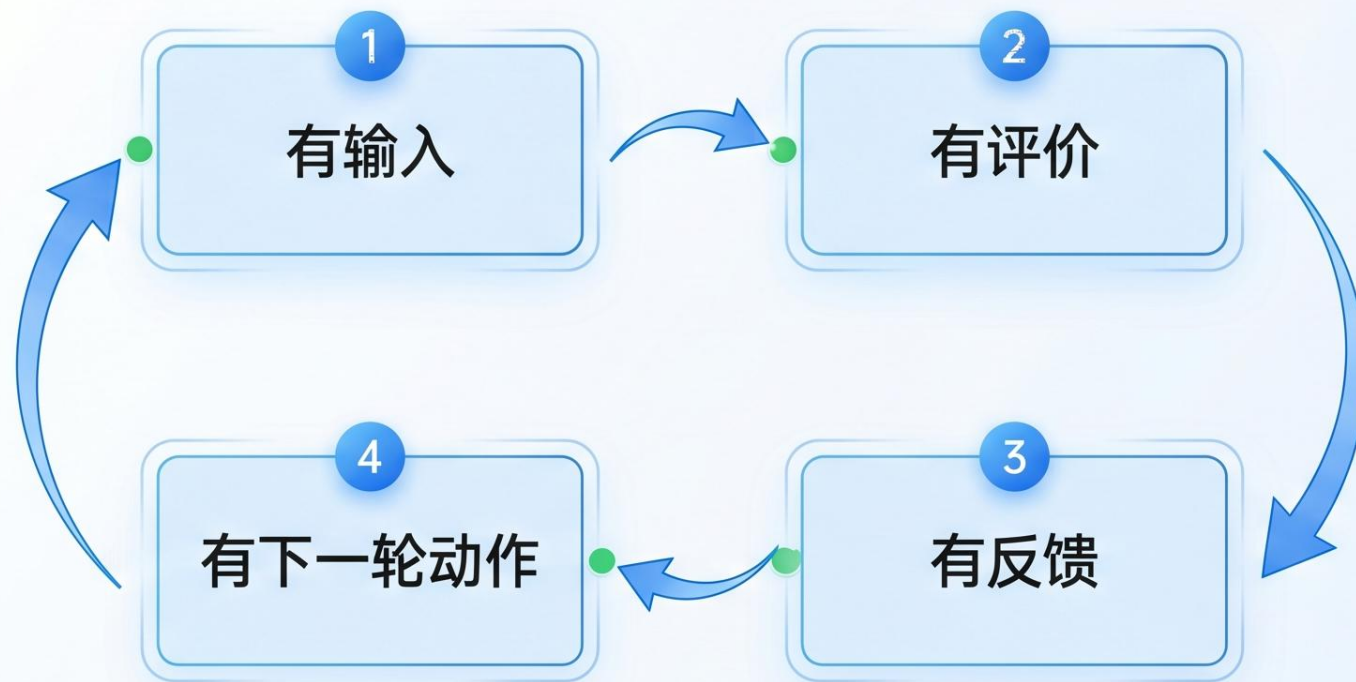
这个任务适合长程 Agent 吗？

第二座山



- 考虑切换成 24 小时轮值工作模式（类似印度+美国模式）

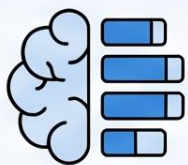
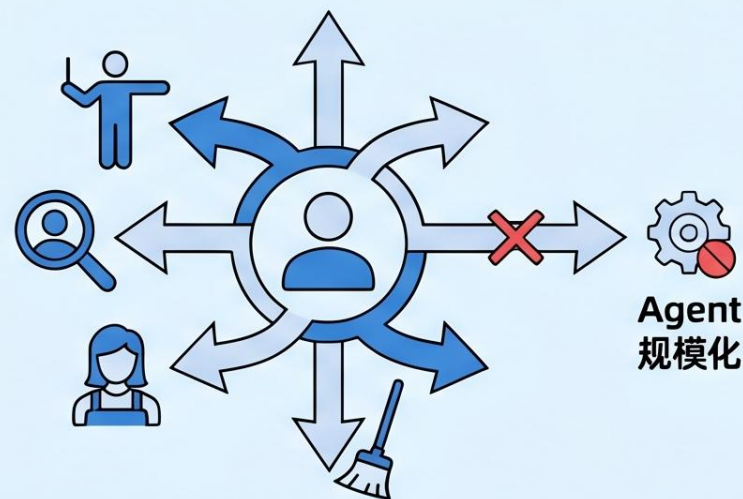
第二座山解法：只挑存在迭代闭环的场景



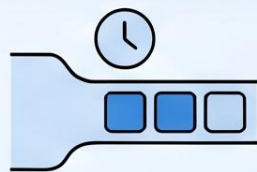
从问答 → 流程 → Agent

第三座山：认知负担，每个人都在当调度器

如果每个人都要自己
当调度器、监工、善后员，
Agent 规模化一定起不来



极端聪明的人，并行的
也只是 7 ± 2 个线程



全部事情都在等你决策



操作少了 $\neq$ 管理成本低了

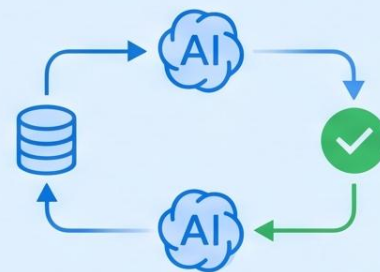
第三座山解法：让 AI 做更多，把问答题变选择题

1. 把需要配合决策的问答题变成选择题



2. 有明确验证闭环的，都闭环给 AI

定时任务 → 主动挖掘任务（‘中国好员工’）



不是人做得更少，而是 AI 主动做得更多

给"非流程停止"装上告警



等待时间 / 人工接力时间
/ 资源排队时间



等待



人工接力



资源排队



统计不考核，保持反思



统计



反思



考核



并发和睡后不是考核指标
，而是反思指标



并发



睡后



反思指标



考核指标

已经做了 1 年多的长程智能体：迭代修复、迭代优化

快速模板

性能劣化定位

开源Bug修复

编译优化探索

参数自动调优

提示词优化

性能优化探索

探索空间

定义参数空间的维度和范围

空间类型

Git版本空间

仓库地址

添加新的工蜂仓库

补充提示词（可借助知识库一键生成）

机器配置：CPU 4核，内存 16G，硬盘 100G；参数

必须设置为100M

一键生成

探索方式

定义如何执行探索任务

执行方式

现场探索

现场探索：使用AI智能优化参数；智研流水线：使用智研平台进行参数

产品配置文件路径

WO

参数配置文件的完整路径

目标服:

优化目标

需要优化的指标

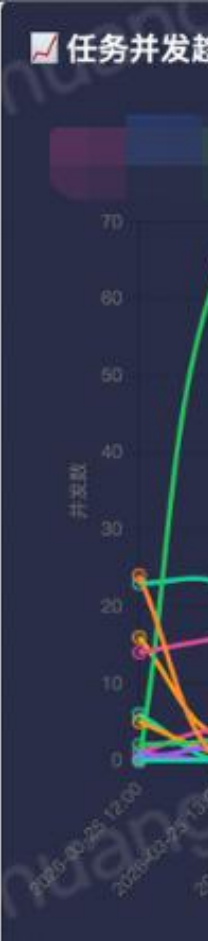
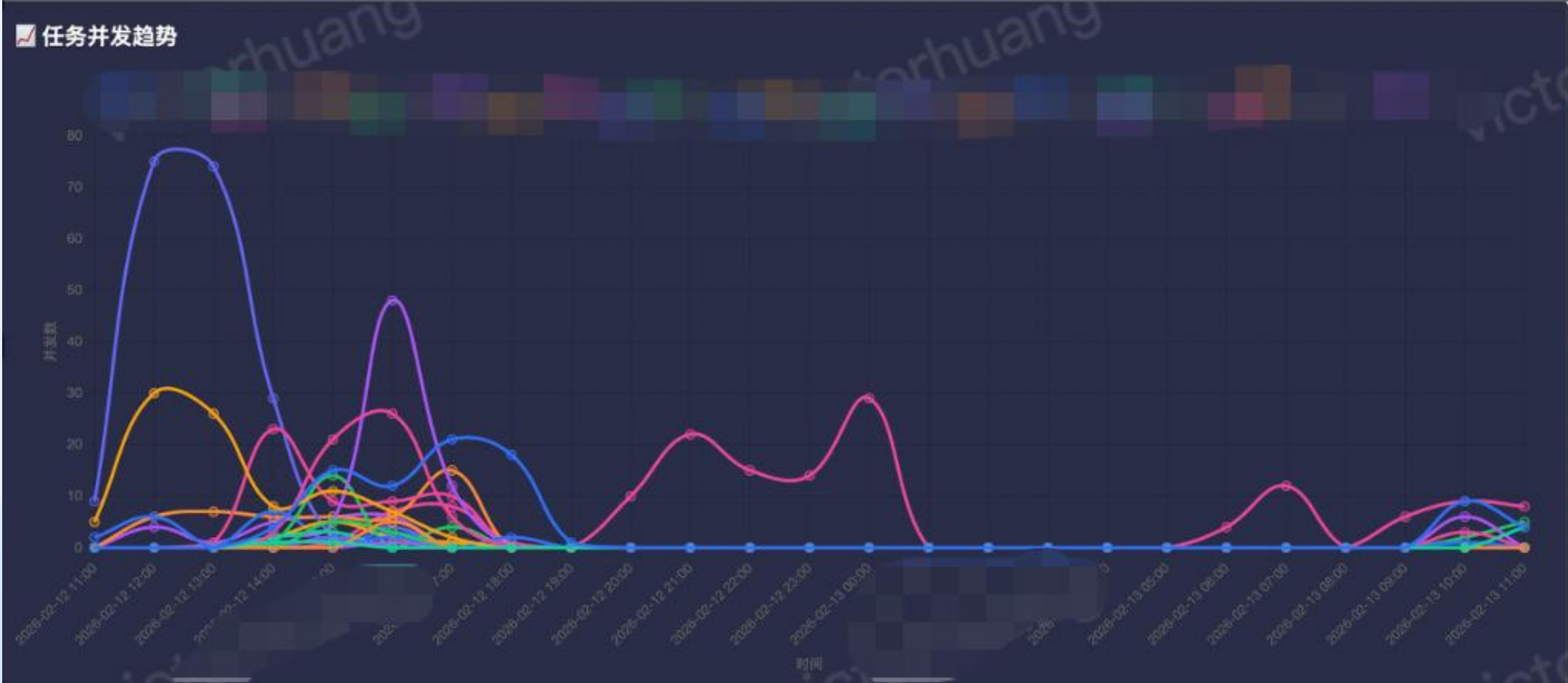
后台会根据 benchmark 步骤的输出进行匹配

服务重启配置

QCon
全球软件开发大会

InfoQ 极客传媒

成效：看看绿色线



小结：一定不能把手段变成目的

手段

- Agent 规模化
- 高并发
- 睡后工作



目的

- 创造价值
- 降低认知负担
- 让团队持续变强



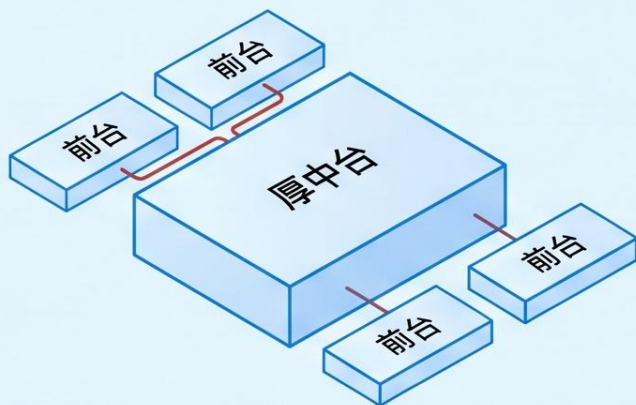
不能作为考核手段，作为反思手段；更关注创造价值



超级团队本质是什么？

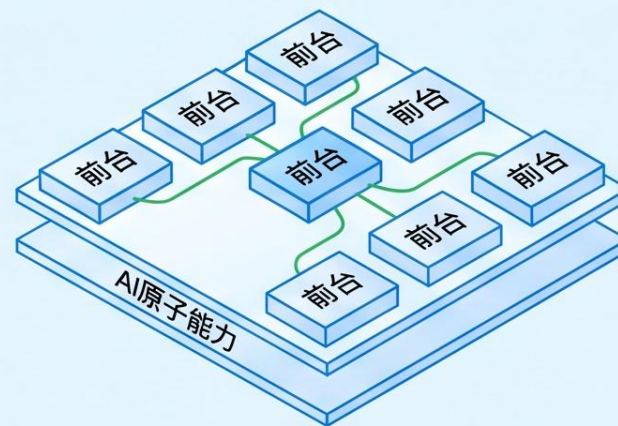
阿米巴（厚中台，薄前台）

- 为了创新建厚中台
- 反而妨碍创新
- 前台缺授权



小麻雀（薄中台，厚前台）

- ✓ AI 提供原子基础能力
- ✓ 战场上的人有更大授权和创新
- ✓ 算力和模型基础足够
- ✓ AI 让薄中台成为可能



1 主动 Agent



一直积极主动

- 先发现、先提醒、先发起

✓ 接住“别让我一直记着”

2 长程 Agent

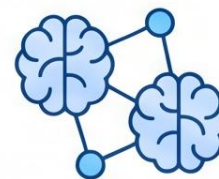


不会累、会自我迭代

- 拆解任务、自动重试

✓ 接住“别让我一直盯着”

3 共脑



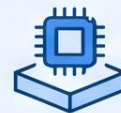
理想态 评估标准 案例 决策树

Agent 和人之间的共享记忆

✓ 接住“别让我每次重讲一遍”



招聘有判断力和更高认知负担上限的人



算力和模型基础

四道坎

1 不去用 → 已翻过



2 用太少 → 改流程、拓目的、增动力



3 不动脑 → 先定义理想态和评估标准



4 太烧脑 → 主动 Agent / 长程 Agent / 共脑



超级团队的本质

- ▶ 薄中台，厚前台
- ▶ 让认知负担被组织接住
- ▶ 定义问题、切问题、验结果
- ▶ 从用 AI 到 AI 自己找活干

判断卡：先看闭环，再谈规模

THANKS

大模型正在重新定义软件

Large Language Model Is Redefining The
Software

